

“Pondo a mão na massa”

A nossa primeira API rest do advpl, vamos fazer um programa para retornar a descrição, unidade de medida e o status de um determinado produto.

Passo 01: Includes

Para criar um webservice REST no Protheus, precisamos da utilização de um include específico, esse include possui alguns DEFINES que facilita muito o desenvolvimento da nossa funcionalidade. Esse include é o **RestFul.CH**.

```
#Include 'Protheus.ch'  
#Include 'FWMVCDEF.ch'  
#Include 'RestFul.CH'
```

```
User Function EREST_01()  
Return
```

Acima definimos uma User Function, porém ela serve apenas com “Dummy Function” (Função fantasma) ou seja, ela serve apenas para reservarmos o nome do nosso “.prw” dentro do repositório, é claro que nada te impede de utilizar essa “Dummy function” como uma função de processamento chamada no nosso programa ADVPL.

Passo 02: Definição da estrutura do Webservice

Para criarmos um webservice, precisamos definir a estrutura do mesmo, ou seja, os atributos e métodos que iremos disponibilizar.

Nesse nosso primeiro exemplo, como citado acima, vamos falar apenas do método GET (método responsável por retornar informações, se estivéssemos tratando de um CRUD esse “cara” seria o Ready).

```
WSRESTFUL PRODUTOS DESCRIPTION "Serviço REST para manipulação de Produtos"
```

```
WSDATA CODPRODUTO As String
```

```
WSMETHOD GET DESCRIPTION "Retorna o produto informado na URL" WSSYNTAX "/PRODUTOS || /PRODUTOS/{}"
```

END WSRESTFUL

Entendendo o trecho de código acima:

Linha 1:

WSRESTFUL -> Início da declaração da estrutura do Webservice;

PRODUTOS -> Nome do webservice

DESCRIPTION -> Descrição do serviço

Logo após definirmos a criação do webservice, deveremos definir quais atributos o mesmo terá.

Linha 3:

WSDATA -> Definição dos atributos

No nosso caso, teremos um único atributo que é o CODPRODUTO e esse atributo é do tipo string, porém ele poderia ser:

- Float
- Boolean
- String
- Array
- Integer

Linha 5:

WSMETHOD -> Definição de um método

GET -> Define qual o tipo do método que estamos criando, podendo ser

- GET
- POST
- UPDATE
- DELETE

DESCRIPTION -> Descrição do método.

SYNTAX -> Sintaxe HTTP da chamada REST. Esta informação é utilizada na documentação do REST.

Linha 6:

END WSRESTFUL -> Encerra a declaração da estrutura do webservice.

Passo 03: Definição do método.

A definição do método é o passo onde vamos desenvolver toda a regra de negócio do método. Como nosso método é um **GET**, deveremos devolver um **JSON** para a aplicação CLIENT. Para a criação do JSON que vamos devolver para o Client, iremos utilizar a função [FWJsonSerialize](#), veremos mais a frente sua utilização.

```
WSMETHOD GET WSRECEIVE CODPRODUTO WSSERVICE PRODUTOS
```

```
Local cCodProd := Self:CODPRODUTO
```

```
Local aArea := GetArea()
```

```
Local oObjProd := Nil
```

```
Local cStatus := ""
```

```
Local cJson := ""
```

```
::SetContentType("application/json")
```

```
DbSelectArea("SB1")
```

```
SB1->( DbSetOrder(1) )
```

```
If SB1->( DbSeek( xFilial("SB1") + cCodProd ) )
```

```
cStatus := If( SB1->B1_MSBLQL == "1", "Sim", "Nao" )
```

```
oObjProd := Produtos():New(SB1->B1_DESC, SB1->B1_UM, cStatus)
```

```
EndIf
```

```
cJson := FWJsonSerialize(oObjProd)
```

```
::SetResponse(cJson)
```

```
RestArea(aArea)  
Return(.T.)
```

Entendendo o trecho de código acima:

Linha 1:

WsMethod -> Indica a iniciação de escrita do método

GET -> Tipo do método (Podendo ser GET, POST, UPDATE e DELETE).

WSRECEIVE -> Indica os parâmetros que iremos receber, no nosso caso, iremos receber o código de produtos via URL (Request Get), mas poderíamos receber também um JSON Object (Request POST).

WSSERVICE -> Indica o nome do webservice ao qual esse método pertence, no nosso caso o nome é PRODUTOS.

Abaixo temos a declaração das variáveis que iremos utilizar, onde no nosso caso, temos 2 que requer nossa atenção, que são as seguintes:

cCodProd -> Armazena o código do produto informado na URL.

oObjProd -> Classe que iremos criar para serializar o nosso json;

Linha 9:

Definição do tipo de retorno:

```
::SetContentType("application/json")
```

Após a definição do tipo de retorno, possuímos um pequeno trecho de código que nos serve para posicionar no produto que recebemos pela URL e iremos serializar o mesmo.

Para serializar o nosso registro JSON, devemos criar uma classe com os atributos que iremos retornar e alimentar os atributos com os valores desejados.

Linha 18:

Após alimentarmos o objeto criado com os atributos que queremos retornar, então devemos utilizar a função FWJsonSerialize que retorna a string do objeto passado como parâmetro.

```
cJson := FWJsonSerialize(oObjProd)
```

Linha 20:

Seta a resposta para a aplicação Client

```
::SetResponse(cJson)
```

Agora vamos ver a nossa aplicação na prática:

Feito toda a codificação do nosso exemplo e compilado no RPO, vamos verificar se o nosso WS Rest está disponível para utilização através do nossa URI:

Microsiga Protheus - RESTFul API

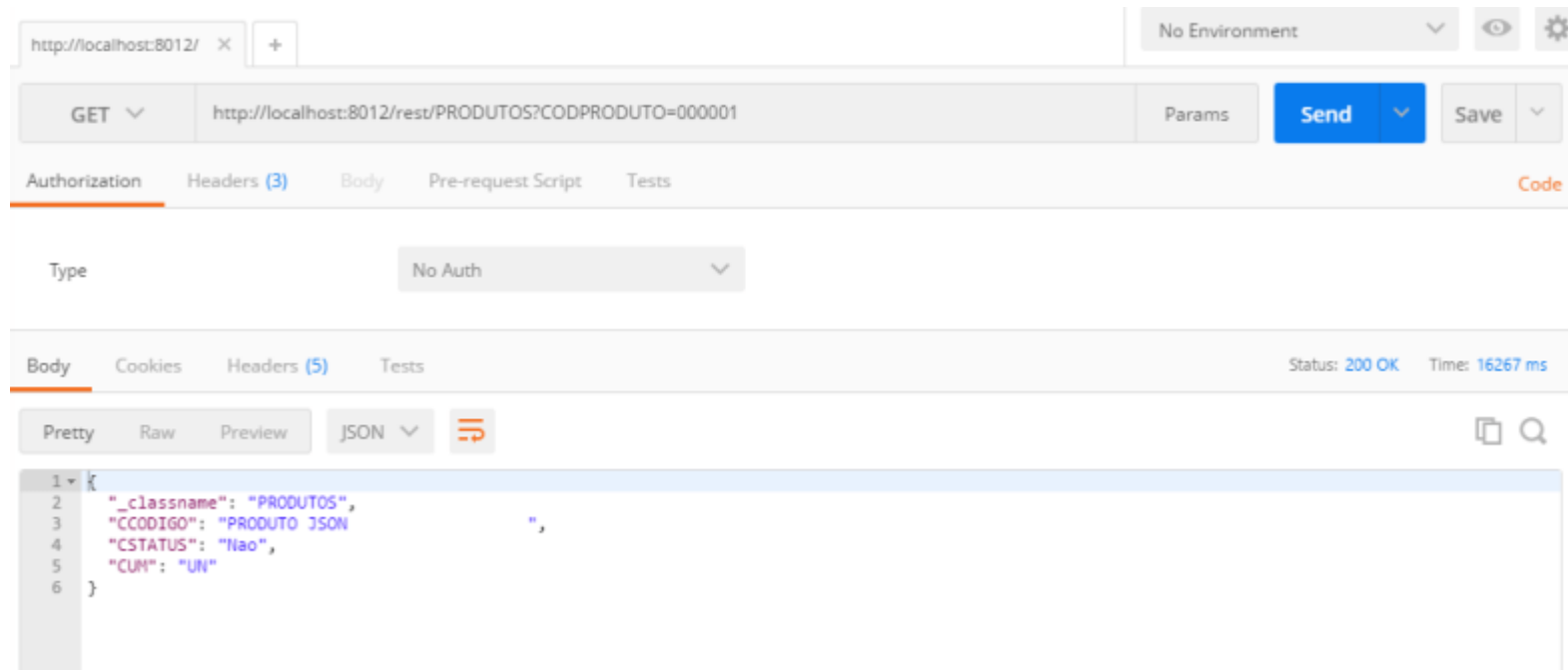
Version 1.15.0717
Path http://localhost:8012/rest
Services 13

API's List

PRODUTOS	/PRODUTOS/ [GET]
Description Serviço REST para manipulação de Produtos For more details	

Agora iremos realizar o “consumo” da API Rest criada através da ferramenta Postman ou no nosso caso como se trata de uma API aberta e sem nenhum tipo de autenticação, também pode ser chamado direto pela URL no navegador:

Teste pelo Postman:



Teste pelo Navegador:



Rest Preparein

Produto:	ADVPL
Versões:	P11-P12
Ocorrência:	Após configuração de INI para uso de webservice Rest, quando é chamado alguma rotina o sistema está considerando uma filial diferente da que está definida na tag PrepareIn Definimos nesta tag que o padrão será a empresa 01 e filial 06 Porém, quando é feito uma busca, o xFilial está retornado a filial 01.
Ambiente:	Todos
Passo a passo:	"A definição do PREPAREIN no REST é diferente do conceito utilizado nos webservices convencionais. Cada valor da chave PREPAREIN representa um grupo de empresas diferente para acesso ao serviço. Por exemplo, a chave PREPAREIN configurada com os valores 19 avisa ao serviços REST que o grupo de empresa 19 vai receber requisições, e deve então ter threads preparadas para isso. Para que todos os grupos possam receber requisição, a chave PREPAREIN deve estar configurada como PREPAREIN=ALL Cada thread é montada com a primeira filial válida para aquele grupo. Para que o valor da filial seja trocada pela camada do REST é necessário enviar, no cabeçalho da requisição REST o parâmetro tenantid, conforme abaixo: tenantid: 19,D MG 02. Neste caso, o servidor REST irá buscar uma thread disponível para o grupo 19 e a execução será realizada na filial D

MG 02. Se o tenantid não for enviado no header da requisição ela será processada na primeira filial válida."

Com essa informação realizamos teste co um serviço simples para retornar a filial que se encontra logada utilizando o fonte abaixo :

```
#include "protheus.ch  
#include "restful.ch"
```

```
WSRESTFUL tstRetFil DESCRIPTION "Serviço REST para verificação da filial logada"
```

```
WSMETHOD GET DESCRIPTION "Retorno da verificação das filiais" WSSYNTAX "/tstRestFil"
```

```
END WSRESTFUL
```

```
WSMETHOD GET WSSERVICE tstRetFil
```

```
// define o tipo de retorno do método  
::setContentType("application/json")
```

```
::setResponse([''])  
::setResponse('{}')  
::setResponse( "Retorno cFilAnt:" + cFilAnt + ';' )  
::setResponse( "Retorno xFilial:" + xFilial("SED"))  
::setResponse('{}')
```



```
::setResponse('')
```

```
return .t.
```

Utilizamos o postman para ser o client rest no processo enviando o header informado ([02. REST com ERP Microsiga Protheus](#))

Método GET

Produto:	ADVPL
Versões:	
Ocorrência:	Gostaria de um material que explicasse realmente como implementar a tecnologia RESTFul no Protheus, com exemplos de APIs, exemplos de programas externos consumindo o serviço, e exemplos de envio e recebimento de dados.
Ambiente:	Todos
Passo a passo:	O método GET é responsável por retornar ao cliente requisitador alguma coisa qualquer, como por exemplo uma lista de clientes cadastrados. Vamos ver abaixo um exemplo de método GET: <pre>WSMETHOD GET WSSERVICE CLIENTES Local aArea := GetArea() Local cNextAlias := GetNextAlias() Local oCliente := CLIENTES():New() // -> Objeto da classe cliente Local oResponse := FULL_CLIENTES():New() // -> Objeto que será serializado</pre>

Local cJSON := ""

Local lRet := .T.

BeginSQL Alias cNextAlias

SELECT A1_COD, A1_LOJA, A1_NOME, A1_END, A1_CGC, A1_CEP, A1_DDD, A1_TEL

FROM %table:SA1% SA1

WHERE SA1.%notdel%

EndSQL

(cNextAlias)->(DbGoTop())

If (cNextAlias)->(!Eof())

While (cNextAlias)->(!Eof())

oCliente:SetCodigo(AllTrim((cNextAlias)->A1_COD))

oCliente:SetLoja(AllTrim((cNextAlias)->A1_LOJA))

oCliente:SetNome(AllTrim((cNextAlias)->A1_NOME))

oCliente:SetCGC(AllTrim((cNextAlias)->A1_CGC))

oCliente:SetCEP(AllTrim((cNextAlias)->A1_CEP))

```
oCliente:SetEnd( AllTrim((cNextAlias)->A1_END ))
oCliente:SetDDD( AllTrim((cNextAlias)->A1_DDD ))
oCliente:SetTel( AllTrim((cNextAlias)->A1_TEL ))
oResponse:Add(oCliente)
(cNextAlias)->( DbSkip() )
EndDo
cJSON := FWJsonSerialize(oResponse, .T., .T.,.F.)
::SetResponse(cJSON)
Else
SetRestFault(400, "SA1 Empty")
lRet := .F.
EndIf
RestArea(aArea)
Return(lRet)

Explicação do trecho de código acima:
```

Conforme [FAQ](#), para serialização de um objeto JSON, precisamos passar um objeto para a função [FWJSONSerialize](#), para isso nas linhas 5 e 6 temos a declaração de duas variáveis que são objetos das classes CLIENTES e FULL_CLIENTES.

CLIENTES é onde armazenamos os atributos do cliente.

FULL_CLIENTES é onde armazenamos uma lista dos objetos da classe CLIENTES

Na sequência fazemos uma query na tabela de clientes (SA1).

Na linha 19 temos a condição (cNextAlias)->(!Eof()) , nesse momento eu verifico se a minha query retornou algum resultado.

Retorno da condicional positivo:

Caso tenha retornado então iremos alimentar os objetos como podemos ver o trecho de código entre as linhas 21 e 35.

Entre as linhas **23** e **30** alimentamos o objeto oCliente com os dados do cliente.

Na linha **32** adicionamos o objeto cliente já alimentado, na lista de clientes oResponse pois será essa lista que iremos serializar.

Após sair do loop onde alimentamos os objetos, utilizamos o seguinte comando para transformar a nossa lista de clientes (oResponse) em uma string JSON serializada e na sequência utilizamos o comando SetResponse para enviar a string ao cliente.

```
cJSON := FWJsonSerialize(oResponse, .T., .T., .F.)
```

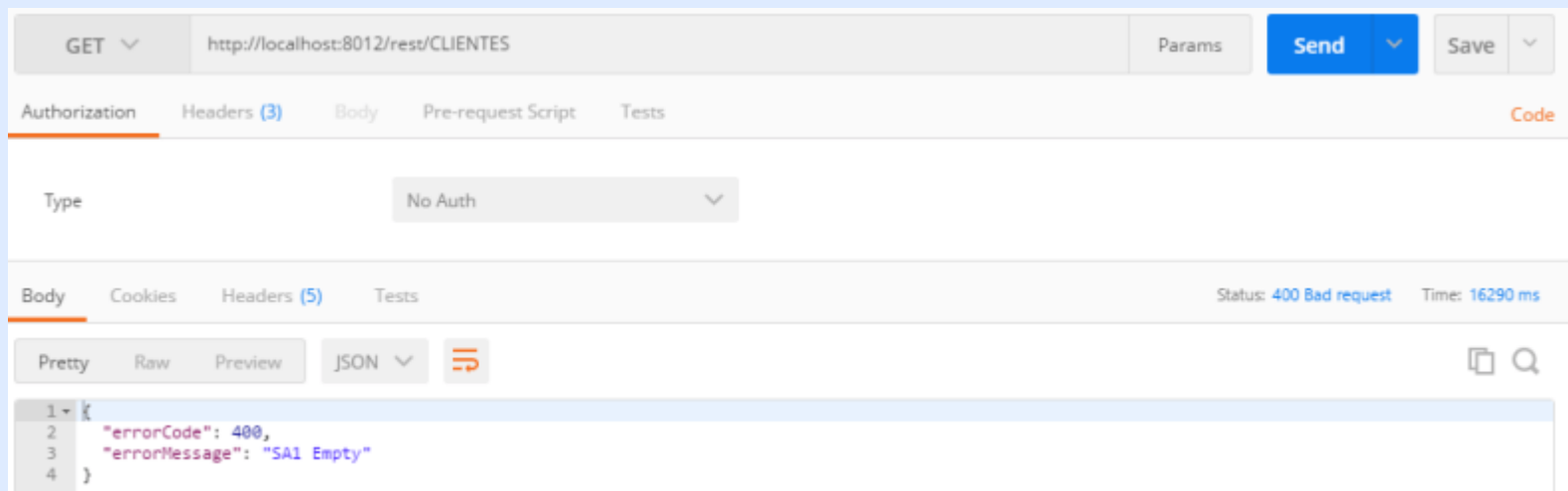
```
::SetResponse(cJSON)
```

Retorno da condicional negativo:

Caso minha query não retorne nenhum registro então iremos utilizar a função SetRestFault para retornar um código HTTP de erro e uma mensagem.

SetRestFault(400, "SA1 Empty")

Exemplo de retorno com o resultado negativo da condicional (sem dados no SA1):



Exemplo de retorno com o resultado positivo da condicional (com dados no SA1):

GET

http://localhost:8012/rest/CLIENTES

Params

Send

Save

Body

Cookies

Headers (5)

Tests

Status: 200 OK

Timer: 5040 ms

Pretty

Raw

Preview

JSON

```
1 {
2   "_classname": "FULL_CLIENTES",
3   "CLIENTES": [
4     {
5       "_classname": "CLIENTES",
6       "CEP": "02469110",
7       "CGC": "",
8       "CODIGO": "000002",
9       "DDD": "",
10      "ENDERECO": "TESTE 0001",
11      "LOJA": "01",
12      "NOME": "CLIENTE TESTE",
13      "TELEFONE": ""
14    },
15    {
16      "_classname": "CLIENTES",
17      "CEP": "02469110",
18      "CGC": "",
19      "CODIGO": "000002",
20      "DDD": "",
21      "ENDERECO": "TESTE 0001",
22      "LOJA": "01",
23      "NOME": "CLIENTE TESTE",
24      "TELEFONE": ""
25    }
26  ]
27 }
```

Método POST

Produto:	ADVPL
Versões:	
Ocorrência:	Gostaria de um material que explicasse realmente como implementar a tecnologia RESTFul no Protheus, com exemplos de APIs, exemplos de programas externos consumindo o serviço, e exemplos de envio e recebimento de dados.
Ambiente:	Todos
Passo a passo:	O método POST é responsável por incluir ou processar algum tipo de informação enviada pelo client. Vamos ver abaixo um exemplo de método post: Iremos receber um JSON com dados de um cliente e realizar a inclusão do mesmo caso não exista na base de dados. Estrutura do JSON que vamos receber:


```
{
  "CLIENTE":{
    "CEP":"00000000",
    "CGC":"75220119737",
    "ENDereco":"TESTE 00001.01",
    "NOME":"CLIENTE 000001.01",
    "TELEFONE":"0000-0000",
    "ESTADO": "SP",
    "MUNICIPIO":"SAO PAULO"
  }
}
```

WSMETHOD POST WSRECEIVE RECEIVE WSSERVICE CLIENTES

Local cJSON := Self:GetContent() // Pega a string do JSON

Local oParseJSON := Nil

Local aDadosCli := {} //--> Array para ExecAuto do MATA030

Local cFileLog := ""

Local cJsonRet := ""

Local cArqLog := ""

Local cErro := ""

Local cCodSA1 := ""

Local lRet := .T.

Private lMsErroAuto := .F.

```
Private IMsHelpAuto := .F.

// -> Cria o diretório para salvar os arquivos de log

If !ExistDir("\log_cli")

MakeDir("\log_cli")

EndIf

// -> Deserializa a string JSON

FWJsonDeserialize(cJson, @oParseJSON)

SA1->( DbSetOrder(3) )

If !(SA1->( DbSeek( xFilial("SA1") + oParseJSON:CLIENTE:CGC ) ))

cCodSA1 := GetNewCod()

Aadd(aDadosCli, {"A1_FILIAL", xFilial("SA1"), Nil} )

Aadd(aDadosCli, {"A1_COD", cCodSA1, Nil} )

Aadd(aDadosCli, {"A1_LOJA", "01", Nil} )

Aadd(aDadosCli, {"A1_CGC", oParseJSON:CLIENTE:CGC, Nil} )

Aadd(aDadosCli, {"A1_NOME", oParseJSON:CLIENTE:NOME, Nil} )

Aadd(aDadosCli, {"A1_NREDUZ", oParseJSON:CLIENTE:NOME, Nil} )
```

```

Aadd(aDadosCli, {"A1_END", oParseJSON:CLIENTE:ENDERECO , Nil}

Aadd(aDadosCli, {"A1_PESSOA", If(Len(oParseJSON:CLIENTE:CGC)= 11, "F", "J") , Nil} )

Aadd(aDadosCli, {"A1_CEP", oParseJSON:CLIENTE:CEP , Nil} )

Aadd(aDadosCli, {"A1_TIPO", "F", Nil} )

Aadd(aDadosCli, {"A1_EST", oParseJSON:CLIENTE:ESTADO , Nil} )

Aadd(aDadosCli, {"A1_MUN", oParseJSON:CLIENTE:MUNICIPIO, Nil} )

Aadd(aDadosCli, {"A1_TEL", oParseJSON:CLIENTE:TELEFONE, Nil} )

MsExecAuto({|x,y| MATA030(x,y)}, aDadosCli, 3)

If lMsErroAuto

cArqLog := oParseJSON:CLIENTE:CGC + " - " + SubStr(Time(),1,5 ) + ".log"

RollBackSX8()

cErro := MostraErro("\log_cli", cArqLog)

cErro := TrataErro(cErro) // -> Trata o erro para devolver para o client.

SetRestFault(400, cErro)

lRet := .F.

Else

```

```
ConfirmSX8()
```

```
cJSONRet := '{ "cod_cli":' + SA1->A1_COD + '";
```

```
+ ',"loja":' + SA1->A1_LOJA + '";
```

```
+ ',"msg":' + "Sucesso" + '";
```

```
+ '}'
```

```
::SetResponse( cJSONRet )
```

```
EndIf
```

```
Else
```

```
SetRestFault(400, "Cliente já cadastrado: " + SA1->A1_COD + " - " + SA1->A1_LOJA)
```

```
lRet := .F.
```

```
EndIf
```

```
Return(lRet)
```

Explicação do trecho de código acima:

Linha 3 -> Pega o JSON recebido e armazena na variável cJSON

Linha 19 -> Deserializa Json recebido utilizando a função FWJsonDeserialize e disponibiliza os dados do mesmo no objeto oParseJSON.

Na linha 22 temos uma condicional que verifica se o CPF/CNPJ recebido já consta em nossa base de dados.

Retorno Negativo da condicional:

Caso o CGC informado não exista na nossa base de dados, então vamos realizar a inclusão do cliente usando ExecAuto.

Montamos o array e chamamos a função MsExecAuto.

Retorno para o Client:

Se no MsExecAuto der algum tipo de inconsistência então retornamos a mensagem de inconsistência para o cliente utilizando a função SetRestFault como podemos ver na linha 51.

Caso contrário eu devolvo um JSON com o código e loja do cliente que acabou de ser incluído, como podemos ver na linha 59.

- Repare que na linha 59 eu estou utilizando o comando SetResponse utilizando uma string que montei “na unha”, como podemos ver no seguinte trecho:

```
cJSONRet := '{"cod_cli":"' + SA1->A1_COD + '"';  
+ ', "loja":"' + SA1->A1_LOJA + '"';  
+ ', "msg":"' + "Sucesso" + '"';  
+ '}'
```

Sim ... Podemos montar o nosso JSON “na unha”, apesar de ser muito mais trabalhoso.

Exemplo de retorno com erro do execauto:

POST http://localhost:8012/rest/CLIENTES Params Send Save

Authorization Headers (3) Body Pre-request Script Tests Code

form-data x-www-form-urlencoded raw binary Text

```
1 {
2   "CLIENTE":{
3     "CEP":"00000000",
4     "CGC":"75220119737",
5     "ENDereco":"TESTE 00001.01",
6     "NOME":"CLIENTE 000001.01",
7     "TELEFONE":"0000-0000",
8     "ESTADO": "SP",
9     "MUNICIPIO":"SAO PAULO"
10  }
11 }
```

Body Cookies Headers (5) Tests Status: 400 Bad request Time: 1559 ms

Pretty Raw Preview JSON

```
1 {
2   "errorCode": 400,
3   "errorMessage": "HELP: OBRIGAT - Um ou mais campos obrigat\u00f3rios n\u00e3o foram preenchidos. xxxxxxxxxxxxxxxxxxxx -> - xxxxxxxxxxxx Pasta -
  Codigo - - "
```

Exemplo de retorno com um cliente já existente na base de dados:

POST http://localhost:8012/rest/CLIENTES

Authorization Headers (3) Body Pre-request Script Tests

form-data x-www-form-urlencoded raw binary Text

```
1 {
2   "CLIENTE":{
3     "CEP":"25389856503",
4     "CGC":"25389856503",
5     "ENDereco":"TESTE 00001.01",
6     "NOME":"CLIENTE 000001.01",
7     "TELEFONE":"0000-0000",
8     "ESTADO": "SP",
9     "MUNICIPIO":"SAO PAULO"
10  }
11 }
```

Body Cookies Headers (5) Tests Status: 400 Bad request

Pretty Raw Preview JSON

```
1 {
2   "errorCode": 400,
3   "errorMessage": "Cliente j\u00e1 cadastrado: 000001 - 01"
4 }
```

Exemplo de retorno com sucesso na inclusão de um cliente:

POST

http://localhost:8012/rest/CLIENTES

Params

Send

Save

AuthorizationHeaders (3)BodyPre-request ScriptTestsCode

form-data

x-www-form-urlencoded

raw

binary

Text

```
1 {
2   "CLIENTE":{
3     "CEP": "00000000",
4     "CGC": "04594148336",
5     "ENDereco": "TESTE 00001.01",
6     "NOME": "CLIENTE 000001.01",
7     "TELEFONE": "0000-0000",
8     "ESTADO": "SP",
9     "MUNICIPIO": "SAO PAULO"
10  }
11 }
```

BodyCookiesHeaders (5)TestsStatus: 201 CreatedTime: 12522 ms

PrettyRawPreviewJSON

```
1 {
2   "cod_cli": "000003",
3   "loja": "01",
4   "msg": "Sucesso"
5 }
```


Método PUT

Produto:	ADVPL
Versões:	
Ocorrência:	Gostaria de um material que explicasse realmente como implementar a tecnologia RESTFul no Protheus, com exemplos de APIs, exemplos de programas externos consumindo o serviço, e exemplos de envio e recebimento de dados.
Ambiente:	Todos
Passo a passo:	O método PUT é responsável por alterar algum tipo de informação enviada pelo client. Vamos ver abaixo um exemplo de método put: Iremos receber um JSON com dados de um cliente e realizar a alteração do mesmo caso exista na base de dados. Neste exemplo teremos parâmetros recebidos via URL que no caso será o CGC (cpf ou cnpj) do cliente que queremos realizar a alteração e o conteúdo via JSON que será recebido via POST-Request (no corpo da requisição HTTP). Estrutura da JSON que iremos receber:

```
{
  "CLIENTE":{
    "ENDERECO":"TESTE 00001.01",
    "NOME":"CLIENTE 000001.01"
  }
}
```

WSMETHOD PUT WSRECEIVE RECEIVE WSSERVICE CLIENTES

Local cJSON := Self:GetContent() // -> Pega a string do JSON

Local cCGC := Self:CGC // -> Pega o parâmetro recebido pela URÇ

Local lRet := .T.

Local oParseJSON := Nil

Local aDadosCli := {} // -> Array para ExecAuto do MATA030

Local cJsonRet := ""

Local cArqLog := ""

Local cErro := ""

Private lMsErroAuto := .F.

If !ExistDir("\\log_cli")

MakeDir("\\log_cli")

EndIf

```

::SetContentType("application/json")

// -> Deserializa a string JSON

FWJsonDeserialize(cJson, @oParseJSON)

SA1->( DbSetOrder(3) )

If (SA1->( DbSeek( xFilial("SA1") + cCGC ) ))

Aadd( aDadosCli, {"A1_NOME", oParseJSON:CLIENTE:NOME , Nil } )

Aadd( aDadosCli, {"A1_END", oParseJSON:CLIENTE:ENDERECO , Nil } )

MsExecAuto({|x,y| MATA030(x,y)}, aDadosCli, 4)

If lMsErroAuto

cArqLog := oParseJSON:CLIENTE:CGC + " - " + SubStr( Time(),1,5 ) + ".log"

cErro := MostraErro("\log_cli", cArqLog)

cErro := TrataErro(cErro) // -> Trata o erro para devolver para o client.

SetRestFault(400, cErro)

lRet := .F.

Else

cJSONRet := '{"cod_cli":' + SA1->A1_COD + '>';

```

```
+ „loja”:”‘ + SA1->A1_LOJA + “”‘;
```

```
+ „msg”.”‘ + “Alterado” + “”‘;
```

```
+’}’
```

```
::SetResponse( cJSONRet )
```

```
EndIf
```

```
Else
```

```
SetRestFault(400, “Cliente não encontrado.”)
```

```
lRet := .F.
```

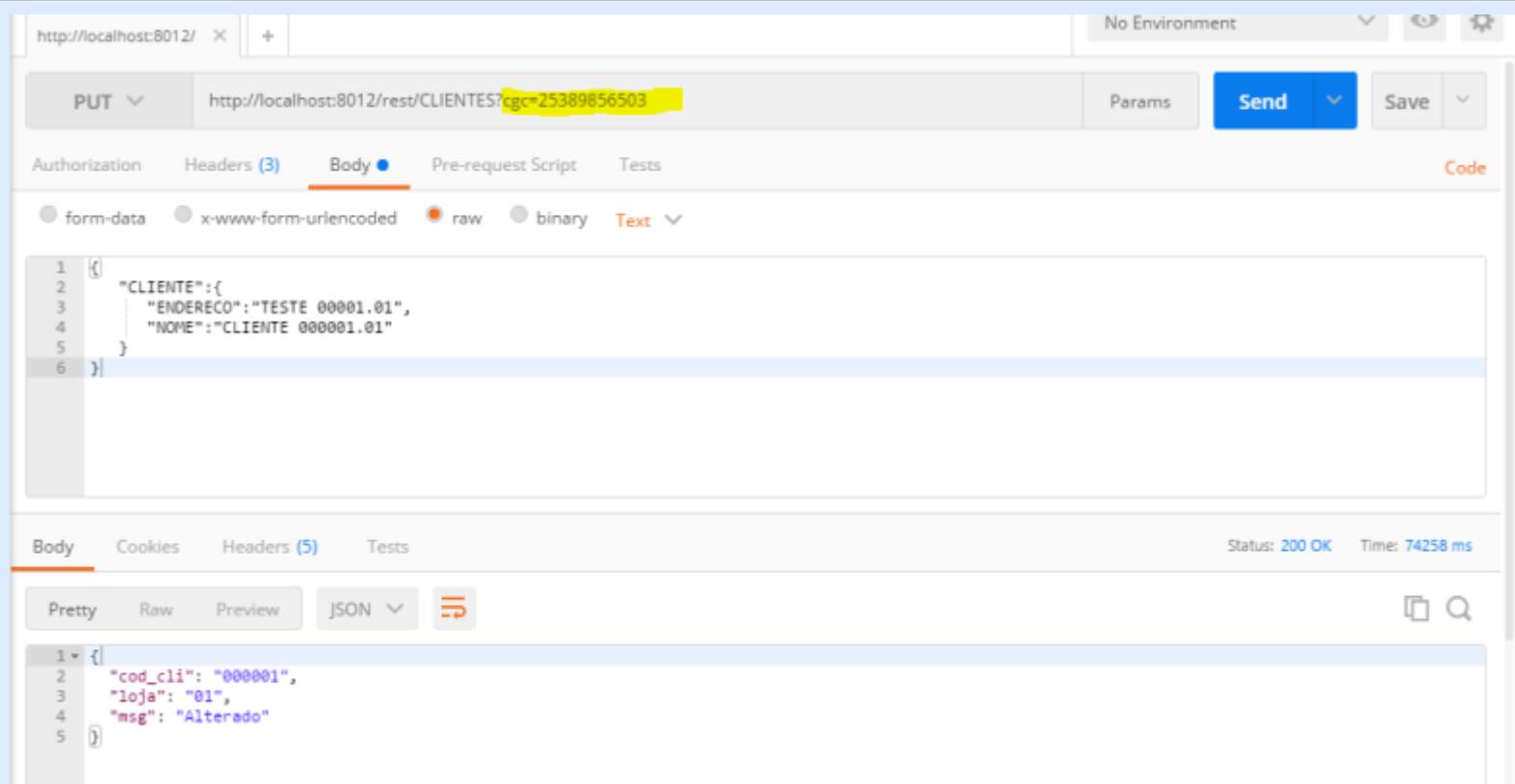
```
EndIf
```

```
Return(lRet)
```

O único ponto de atenção que deveremos ter nesse método é a linha 4, pois é ali que recebemos o CGC do cliente informado através da URL no qual iremos modificar.

Todo o restante dos comandos, já está explicado nos métodos [POST](#) e [GET](#).

Exemplo de requisição efetuada com sucesso:



Exemplo de requisição efetuada com erro:

http://localhost:8012/ x + No Environment

PUT http://localhost:8012/rest/CLIENTES?cg=253898565031 Params Send Save

Authorization Headers (3) Body Pre-request Script Tests Code

☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary Text

```
1 {
2   "CLIENTE":{
3     "ENDERECO": "TESTE 00001.01",
4     "NOME": "CLIENTE 000001.01"
5   }
6 }
```

Body Cookies Headers (5) Tests Status: 400 Bad request Time: 101 ms

Pretty Raw Preview JSON

```
1 {
2   "errorCode": 400,
3   "errorMessage": "Cliente n\u00f3 encontrado."
4 }
```